ACTIVITY NO. 7	
SORTING ALGORITHMS: BUBBLE, SELECTION AND INSERTION SORT	
Course Code: CPE010	**Program:** Computer Engineering
Course Title: Data Structures and Algorithms	**Date Performed:** October 16, 2024
Section: CPE010-21S1	**Date Submitted:**
Name(s): Tankeko, Ma. Bea Angela L.	**Instructor:** Engr/ Rizette Sayo

6. Output

ILO A: Create A C++ code for implementation of selection, bubble, insertion, and merge sort.

Creation of code with random values: (Maximum 100):

Code:

```
#include <iostream>
#include <cstdlib>
#include <ctime>
const int max_size = 100;
int main() {
int dataset[max_size];
srand(time(0));
for (int i = 0; i < max_size; i++) {
dataset[i] = rand()%100;
}
for (int i = 0; i < max_size; i++) {
std::cout << dataset[i] << " ";
}
std::cout << std::endl;
return 0;
}
```

Output:

```
/tmp/BxTY9tS72E.o
60 28 77 64 14 18 30 87 93 42 71 69 48 52 59 25 40 38 47 80 58 2 6 48 31 28
    58 53 38 62 84 99 90 61 15 56 79 45 43 25 87 14 94 88 18 53 13 59 43 61
    91 53 15 97 54 99 25 12 52 64 26 88 15 68 50 30 25 81 76 68 6 15 35 1
    55 5 54 69 16 50 82 7 3 97 56 57 48 82 22 1 98 48 89 13 69 91 95 94 73
    23


=== Code Execution Successful ===
```

***A.1*-Bubble Sort Technique:**

Code:

```
#include <iostream>
#include <cstdlib>
#include <ctime>
const int max_size = 100;
template <typename T>
void bubbleSort(T arr[], size_t arrSize){
        for (size_t i = 0; i < arrSize - 1; i++){
        for (size_t j = 0; j <arrSize - i -1; j++){
        if (arr[j] > arr[j + 1]){
        std::swap(arr[j], arr[j + 1]);
        }
        }
        }
}
int main() {
int dataset[max_size];
srand(time(0));
for (int i = 0; i < max_size; i++) {
dataset[i] = rand()%100;
}
bubbleSort(dataset, max_size);
for (int i = 0; i < max_size; i++) {
std::cout << dataset[i] << " ";
}
std::cout << std::endl;
return 0;
}
```

Output:

```
/tmp/3JV3lXkkMk.o
0 0 1 3 3 3 4 8 8 9 10 11 12 12 13 13 16 16 16 18 19 19 20 20 22 22 23 24
    25 26 26 27 29 30 34 36 36 37 38 39 40 40 41 43 43 43 46 46 50 50 52 52
    54 55 56 56 57 57 60 62 64 64 64 64 66 68 69 69 69 70 71 73 74 74 75 76
    79 80 80 80 80 81 83 84 84 84 85 85 86 86 88 90 91 91 92 92 93 95 96 96


=== Code Execution Successful ===
```

Upon the implementation of the provided general algorithm, the randomly generated dataset is now sorted starting from the smallest integer to the largest. Bubble sorting allows the program to compare two values in a dataset and swap following the assigned order.

A:2 Selection Sort Algorithm
//Selection Sort(Array, Size of Array)//

Code:
```
#include <iostream>
#include <cstdlib>
#include <ctime>
const int max_size = 10;
template <typename T>
int Routine_Smallest(T A[], int K, const int arrSize) {
	int minPos = K;
	for (int i = K + 1; i < arrSize; i++) {
	if (A[i] < A[minPos]) {
	minPos = i;
	}
	}
	return minPos;
}
template <typename T>
void selectionSort(T arr[], const int N) {
	for (int i = 0; i < N; i++) {
	int POS = Routine_Smallest(arr, i, N);
	T temp = arr[i];
	arr[i] = arr[POS];
	arr[POS] = temp;
	}
}

int main() {
	int dataset[max_size];
	srand(time(0));
	for (int i = 0; i < max_size; i++) {
	dataset[i] = rand() % 100;
	}
	selectionSort(dataset, max_size);
	for (int i = 0; i < max_size; i++) {
	std::cout << dataset[i] << " ";
	}
	std::cout << std::endl;

	return 0;
}
```

Output:

```
/tmp/0ZXaNrmOoh.o
9 13 16 19 30 31 72 75 83 87
```

In this example, Selection sort checks and finds the smallest value in the array and places it in the correct index to sort the data. In this sorting technique, the program assigns a minimum value and checks if the number succeeding is smaller. If the number succeeding it is smaller, the program assigns that as the minimum value and swaps position, if not it remains as the smallest value and proceeds to compare with another value in the array until it reaches the last value.

//Routing smallest (Array, Current Position, Size of Array)
Code:

A.3 Insertion Sort Algorithm
Code:

```
#include <iostream>
#include <cstdlib>
#include <ctime>
const int max_size = 100;
template <typename T>
void insertionSort(T arr[], const int N){
        int K = 0, J, temp;
        while(K < N){
        temp = arr[K];
        J= K-1;
        while(temp <= arr[J]){
        arr[J+1]= arr[J];
        J--;
        }
        arr[J+1] = temp;
        K++;
        }
}
int main() {
int dataset[max_size];
srand(time(0));
for (int i = 0; i < max_size; i++) {
dataset[i] = rand()%100;
}
insertionSort(dataset, max_size);
        for (int i = 0; i < max_size; i++) {
        std::cout << dataset[i] << " ";
}
std::cout << std::endl;
return 0;
}
```

OUTPUT:

```
/tmp/n32lTjjQss.o
1 1 3 4 5 6 6 7 8 9 11 11 13 14 17 18 20 20 21 22 23 24 24 25
    25 25 27 31 31 32 33 33 34 34 36 38 41 43 43 43 44 45 45
    46 47 50 51 51 53 54 55 55 56 56 57 58 58 58 59 59 61 63
    63 64 64 66 67 68 68 68 70 73 73 74 74 76 78 79 81 82 84
    84 84 84 85 85 87 88 88 90 91 91 92 92 94 95 96 98 98 98
```

7. Supplementary Activity

8. Conclusion

9. Assessment Rubric